Modification d'un entrepôt de données

Gattano César

5 octobre 2026

Avant-propos

Ce rapport a été rédigé en usant de l'intelligence artificielle. Par soucis de transparence, les parties purement rédigées par l'IA sont signalées via une couleur de police bleu sombre. L'auteur certifie que l'ensemble du contenu générée par l'IA a été revu et validé par ses soins.

Voici un exemple : Je suis un assistant IA conçu pour vous aider dans la rédaction de votre rapport. Je génère du texte structuré et cohérent, que vous pouvez ensuite affiner. Mon rôle est de vous fournir un cadre de travail, tout en vous laissant le contrôle final. Ensemble, nous pouvons créer un document clair et professionnel.

En outre, l'auteur indique que l'IA générative a également été utilisée en auto-complétion durant la rédaction de ce rapport. Cette fonctionnalité ayant été utilisée tout au long du rapport, aucun indicateur n'a été mis en place pour souligner cet aspect, si ce n'est cette déclaration.

Table des matières

1 Contexte **2**

2 Préparation de la maintenance **2**
2.1 État initial de l'entrepôt de données 2
2.2 Planification de la maintenance 3
2.3 Suivi du service 3
2.4 Backup régulier 4
2.4.1 Procédure de sauvegarde 4
2.4.2 Procédure de restauration 6
2.4.3 Difficultés rencontrées 6

3 Opération de maintenance **6**
3.1 DBT 6
3.2 Intégration d'une nouvelle source de données 7
3.3 Modèle de staging des données brutes 7
3.4 Création de la table des faits associés aux nouvelles données 8
3.5 Mise à jour de la documentation DBT 8

1 Contexte

Je suis data engineer junior au sein de ShopFlow, une ESN spécialisée dans les solutions data pour le secteur e-commerce. Mon client, la société VenteRapide, exploite une boutique en ligne généraliste présente sur le marché français depuis 2021.

VenteRapide dispose d'un entrepôt de données (data warehouse) opérationnel, construit sur `PostgreSQL`et alimenté via `DBT`. Cet entrepôt centralise les données de commandes, de produits, de clients et de visites du site, et est utilisé quotidiennement par l'équipe analytique pour ses reportings.

Dans le cadre d'une évolution de l'activité, VenteRapide me confie la mission de maintenir et faire évoluer cet entrepôt en conditions opérationnelles.

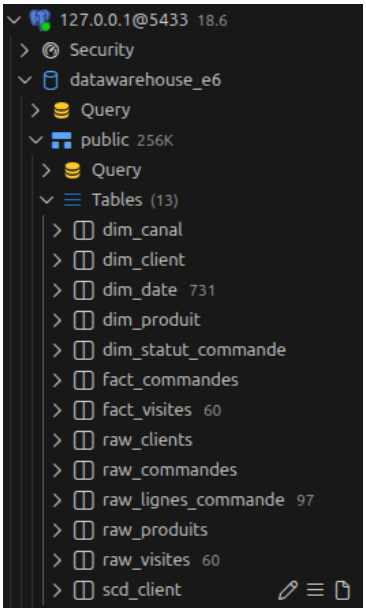


FIGURE 2 – Entrepôt initial

2 Préparation de la maintenance

2.1 État initial de l'entrepôt de données

L'entrepôt initial contient :

— 5 tables de données brutes : `clients`, `commandes`, `lignes_commandes`, `produits`, `visites`,

— 5 tables de dimensions : `canal`, `client`, `date`, `produit`, `statut_commandes`,

— 2 tables de faits : `commandes`, `visites`.

— 1 table SCD type 2 : `client`.

Le linéage des données est assuré par `DBT`, qui orchestre les transformations et les chargements. La figure 1 montre le linéage des données dans l'entrepôt initial.

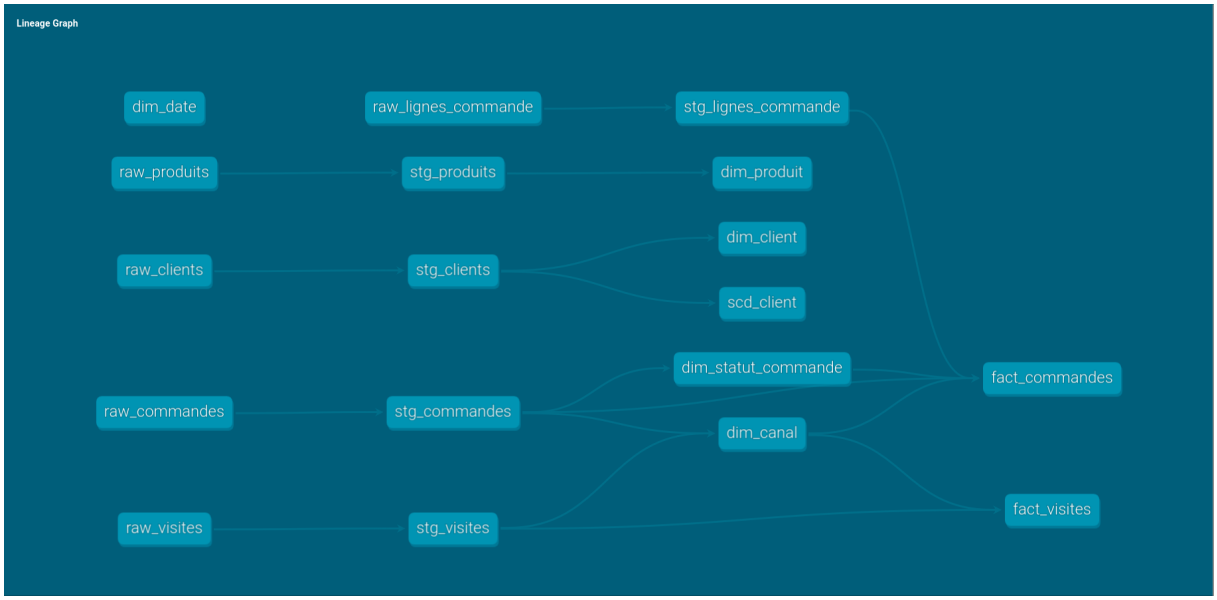
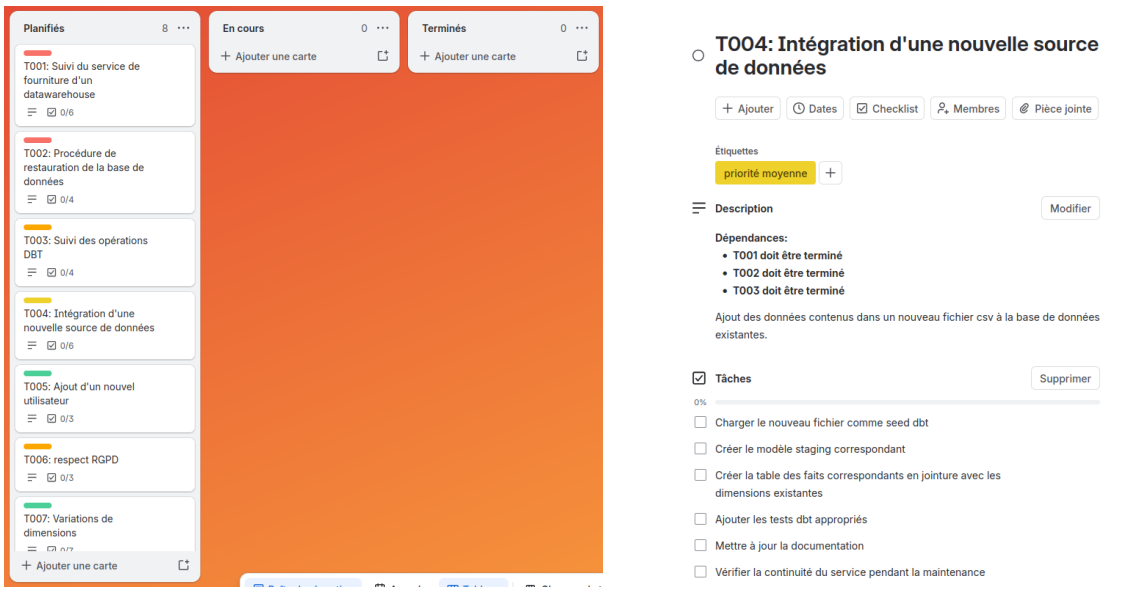


FIGURE 1 – Linéage des données dans l'entrepôt initial

2.2 Planification de la maintenance

La maintenance de l'entrepôt de données est planifiées grâce à un système de tickets. La vue d'ensemble et un exemple de tickets sont présentés en figure 3



(a) Vue du tableau (b) Exemple de ticket de maintenance

FIGURE 3 – Système de tickets de maintenance.

2.3 Suivi du service

Un SLA (Service Level Agreement) est un engagement formel entre un fournisseur de service et ses utilisateurs, définissant le niveau de qualité de service attendu, mesurable par des indicateurs précis, les SLI (Service Level Indicators). Il répond à trois questions : quoi (quel service), combien (quel niveau de performance), et que se passe-t-il en cas de non-respect.

Dans le cas d'une base de données, un SLA pourrait inclure des engagements sur :

— La disponibilité du service sur une période donnée : par exemple, un engagement de 99,9% de disponibilité mensuel.
— Le temps de traitement des incidents : délai de prise en charge et délai de résolution pouvant être fonction du niveau de criticité
— La planification de fenêtre de maintenance : plages de temps où la contrainte de disponibilité peut être levée.
— Recovery point objective (RPO) et Recovery time objective (RTO) : les indicateurs précisant la quantité maximale de perte de données et le temps maximal de restauration acceptable, en cas de rupture de service.
— Performance : temps de réponse des requêtes, nombre de transactions par seconde, etc.

Dans le cadre de ce projet, je fais le choix de mettre en place les SLI suivants :

— Mesure (booléenne) de la santé du conteneur `PostgreSQL` : simple tentative de connexion régulière pour vérifier que la base de données est accessible.
— Mesure de la latence d'une requête simple sur les tables de faits de la base de données (tables de haute valeur pour les équipes métiers).

Le script `scripts/set_monitoring_db.sh` permet de créer la base des données de monitoring et l'utilisateur associé, ainsi que les droits nécessaires via des requêtes SQL envoyés au sein du conteneur `PostgreSQL`. Le script `monitor.py` contient le code de sur-

veillance de la base de données qui écrit à intervalle régulier les mesures décrites ci-dessus dans une table. Enfin le script `streamlit/monitor_viz.py` permet de visualiser ces mesures de latence à l'aide d'un tableau de bord streamlit (figure 4).
Lien utile :

- `https://www.ibm.com/fr-fr/think/topics/service-level-agreement`
- `https://sre.google/sre-book/service-level-objectives/`

Pour aller plus loin :

- Mise en place d'une architecture "collecteur" + "visualiseur" en solution industriel et gratuite grâce à `Prometheus` et `Grafana`.
- Mise en place d'un système d'alerte structuré sur une fenêtre de confirmation (ex : seuil d'une métrique dépassée sur plusieurs mesures consécutives ou p95 supérieur à un seuil sur les 5 dernières minutes.) Technologies possibles : `SQL`+ `smtplib` ou `Prometheus` Alertmanager + `Grafana` Alerting.

2.4 Backup régulier

La stratégie de back-up mis en place repose sur les outils `pg_dump` et `pg_dumpall` (pour la sauvegarde) et `psql`/`pg_restore` pour la restauration.

Il faut distinguer deux types de backup, (1) le backup complet qui reconstruira l'environnement de production complet, roles, privilèges, bases de données, etc... dans sa globalité, et (2) le back-up partiel qui va sélectionner les entités particulières de la base `PostgreSQL` à sauvegarder :

- `pg_dump` tombe dans la catégorie back-up partiel puisqu'elle va permettre de sauvegarder les données d'une base de données (mais ne considèrera pas par exemple la création des rôles ayant des droits sur cette table, ce sont des entités globales),
- `pg_dumpall` tombe dans la catégorie back-up complet (bien que certaines options permettent de limiter son périmètre).

2.4.1 Procédure de sauvegarde

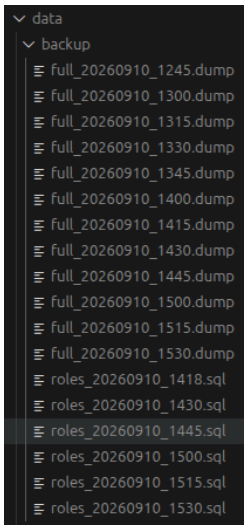


FIGURE 5 – Backup 15min, rétention 3h

En guise de stratégie de sauvegarde et pour expérimenter les deux approches, nous allons procéder comme suit :

- Procédure de sauvegarde complète que l'on restreint à l'extraction des rôles et privilèges, avec `pg_dumpall`
- Procédure de sauvegarde partielle de la seule base de données `datawarehouse_e6`, avec `pg_dump`

Un premier script `scripts/set_backup_roles.sh` permet la première sauvegarde et un deuxième `scripts/set_backup_db.sh` permet la seconde. À savoir qu'il existe deux formats possibles pour les fichiers-dump : ficher-sql contenant les commandes `SQL` pour reconstruire la sauvegarde à l'identique (via `pgsql`) ou un format dédié compréssé (grâce à `zlib`) utilisable par `pg_restore` pour la reconstruction.

Ces scripts permettent également d'opérer la période de rétention des fichiers en appliquant le temps de conservation des fichiers-sql/dump.

C'est ensuite un `cron` qui permet la régularité des opérations de back-up via la planification de l'exécution de ces deux scripts.

Pour l'exercice d'apprentissage, une éxecution toutes les 15 minutes avec un période de rétention de 3 heures a été l'option choisie. En situation opérationnelle, une exécution quotidienne serait plus adaptée avec une retention d'une semaine ou d'un mois.



FIGURE 4 – Screenshot du tableau de bord de monitoring de la base de données

2.4.2 Procédure de restauration

La stratégie de restauration que nous allons voir va permettre de reproduire la base de données `datawarehouse_e6` tout en conservant les accès et privilèges des différents utilisateurs. Afin de tester la procédure, un conteneur jetable sera mis en place afin que les deux tables (originale et restaurée) puissent co-exister.

Afin d'assurer le succès de l'opération, il est nécessaire de reproduire les rôles dans le conteneur dédié à la restauration, puis de restaurer la base de données `datawarehouse_e6` (l'ordre est important). Le script `scripts/test_backup_db.sh` opère cela après la mise en place d'un conteneur.

Vient ensuite une phase de validation avec le lancement de plusieurs commandes SQL pour afficher certaines métriques :

— Liste des rôles restaurés
— Liste des droits sur la table `fact_commandes`
— Nombre d'entrées dans chaque table

Ces sorties sont à comparer avec les mêmes commandes exécutées sur le conteneur original pour s'assurer qu'il n'y a aucune différence.

Enfin, les tests `DBT` sont une validation supplémentaire, plus approfondie permettant de s'assurer qu'il n'y a pas eu de corruption des données, ou de perte de données entraînant une violation des règles mises en place dans les modèles des tables et de leurs relations.

2.4.3 Difficultés rencontrées

— La base de données `PostgreSQL` tournant dans un conteneur `Docker`, les commandes sont envoyés via un 'docker exec'. Certaines options comme '-t' peuvent interférer avec la sortie de `pg_dump` et corrompre le fichier-dump créé au point que la restauration échoue.
— L'utilisation de `cron`nécessite de s'assurer des droits d'éxecution des différents scripts pour la catégorie 'other' ainsi que de l'utilisation de chemin absolue tout au long du script.

Liens utiles :

— `https://www.postgresql.org/docs/current/backup.html`
— `https://sre.google/sre-book/data-integrity/`

Pour aller plus loin :

— `pgBackRest` permet la mise en place d'un back up incrémental ou différentiel
— Pour une stratégie de back-up robuste, la règle d'or est 3-2-1 : 3 copies sur 2 supports différents, 1 support hors site
— Planification de retention sophistiquée : 14 backups des derniers jours + 1 backup par mois des 6 derniers mois

3 Opération de maintenance

3.1 DBT

`DBT` s'intègre dans l'approche ELT qui se démocratise avec l'avénement du cloud et en particulier des technologies de datawarehouse et la réduction du coût de stockage qui permet de ne pas dépendre de machine on-premise et de leur gestion. Cette approche déploie la puissance d'un datawarehouse pour la transformation en les chargeant brutes au préalable. Les données sont plus facilement et rapidement gérable. `DBT` fournit une solution de transformation de données propre et prête pour l'analyse, en poussant l'automatisation et la collaboration sur la modélisation et le traitement des données. Le principe derrière `DBT`et de reproduire les

bonnes pratiques en développement d'application pour le développement de modèle analytique de données.

3.2 Intégration d'une nouvelle source de données

Le fichier `raw_retours.csv` a été ajouté aux `seeds DBT`, pour l'intégrer comme une nouvelle source de données. En parallèle, un fichier `schema.yml` a été générée pour fournir la description de tous les fichiers-csv présent, à des fins de documentation.

En suite la commande :

```
dbt seed
```

va s'emparer du nouveau fichier-csv et créer une table dans la base de données

3.3 Modèle de staging des données brutes

Le modèle de staging selon `DBT`, est un modèle ou peu de transformations sont opérées à partir des données des sources. Il est recommandé de les matérialiser en vues plutôt qu'en table et servent essentiellement à renommer et caster les colonnes et à effectuer des tests sur ces données pour les normaliser (unicité, non-null, valeurs autorisées, ...)

Dans `models`, un fichier `stg_retours.sql` a été créé pour modéliser la table de staging des données de retours.

```
with source as (
    select * from {{ ref('raw_retours') }}
),

renamed as (
    select
        retour_id::integer,
        command_id::integer,
        produit_id::integer,
        client_id::integer,
        date_retour::date,
        motif,
        montant_rembourse::numeric(10,2)
    from source
)
```

En complément, le fichier `schema.yml` a été complété pour inclure la documentation et les tests d'unicité, non-null et de valeurs autorisés sur les différents champs de la table `retours`.

Pour finir la commande :

```
dbt build --select stg_retours
```

Permet la matérialisation de la vue et l'exécution des tests programmés dans le fichier `schema.yml`

Difficultés rencontrées

Au moment de réaliser ce projet, l'équipe `DBT` est en phase de transition vers `DBT` v2 avec le déploiement de `dbt-fusion` en remplacement de `dbt-core`, mais le connecteur `PostgreSQL` n'est pas encore disponible pour cette version. Bien que la v2 soit encore en pré-release la documentation et l'extension VSCode repose sur cette dernière, ce qui a créé une incompatibilité à gérer au niveau de l'installation.

Liens utiles

— DBT fundamentals (VSCode) : `https://learn.getdbt.com/learn/course/dbt-fundamental analytics-development-lifecycle-30min/dbt-and-the-adlc?page=2`

Pour aller plus loin

— Ajout de tests dans le `schema.yml` du dossier `seeds` pour détecter prématurément des fichiers-csv corrompus
— Explorer Fivetran ! pour la partie EL si les sources sont externes
— Recensement des sources dans un fichiers `sources.yaml` si elles sont externes.

3.4 Création de la table des faits associés aux nouvelles données

La table des faits à intégrer respecte un schéma en étoile déjà en place au sein de DBT. Ainsi, la table des faits `fact_retours` et doit être reliés aux tables de dimensions existantes.

Dans `models/facts`, un fichier `fact_retours.sql` a été créé pour modéliser la table. Cette fois, la table est matérialisée comme une table car elle est destinée à être interrogée par les équipes métiers et analystes. Entre autres, ce fichier contient les jointures de tables entre deux modèles de staging et les tables de dimensions existantes.

```
select
    r.retour_id,
    r.commande_id,
    to_char(c.date_commande, 'YYYYMMDD')::integer   as date_commande_id,
    to_char(c.date_expedition, 'YYYYMMDD')::integer as date_expedition_id,
    to_char(c.date_livraison, 'YYYYMMDD')::integer   as date_livraison_id,
    to_char(r.date_retour, 'YYYYMMDD')::integer   as date_retour_id,
    dc.client_id,
    dp.produit_id,
    r.motif,
    l.montant_ligne as montant_paye,
    r.montant_rembourse,
    case
        when c.date_livraison is not null
        then (r.date_retour - c.date_livraison)
        when c.date_commande is not null
        then (r.date_retour - c.date_commande)
        else null
    end as delai_retour_jours
from retours r
left join commande c       on r.commande_id = c.commande_id
left join lignes l         on r.commande_id = l.commande_id and r.produit_id = ...
    l.produit_id
left join dim_client dc    on r.client_id = dc.client_id
left join dim_produit dp   on r.produit_id = dp.produit_id
```

En complément le fichier `schema.yml` a été complété pour inclure la documentation, les tests d'unicité, non-null et de valeurs autorisées ainsi que les relations avec les tables de dimension.

Pour finir la commande :

```
dbt build --select fact_retours
```

permet la matérialisation de la table et l'exécution des tests programmés dans le fichier `schema.yml`.

3.5 Mise à jour de la documentation DBT

Comme mentionné précédemment, la première documentation DBTest intégré aux fichiers `schema.yml`. Elles ont étaient complétées à chaque étape de la maintenance.

Difficultés rencontrées

— Les tests `DBT` ont révélé que certains champs étaient `NULL`. Notamment, les champs date d'expédition et date de livraison, laissant penser qu'il y a peut-être qu'une mode de retrait existe.
— Dans une autre mesure une donnée avec un `montant_paye` nul semble être de l'ordre de l'erreur.